# Engineering Optimization for Matrix Multiplication on GPU


## Abstract

Matrix multiplication is one of the most widely used operations in modern computing. It plays an important role in Artificial Intelligence (AI), Machine Learning (ML), deep learning, scientific computing, and image processing. However, multiplying large matrices requires a significant number of computations and memory operations, making execution speed and computational efficiency important challenges.

This project aims to develop an optimized matrix multiplication implementation using GPU parallel processing and CUDA C++. The main objective is to improve the performance of matrix multiplication by efficiently utilizing GPU resources and optimizing the way data is accessed, stored, and processed.

The project will begin with a basic CPU implementation and a simple GPU implementation to establish performance baselines. Subsequently, several optimization techniques will be applied to improve the GPU implementation. **Matrix Tiling** will divide large matrices into smaller blocks, allowing the GPU to process smaller portions efficiently. **Shared Memory Optimization** will store frequently reused matrix data in faster on-chip memory, reducing repeated accesses to global memory. **Memory Coalescing** will organize memory accesses so that neighboring GPU threads can access neighboring memory locations efficiently. **Thread-Block and Tile-Size Tuning** will experiment with different configurations to improve parallel execution and GPU resource utilization. **Register Optimization** will improve the handling and reuse of intermediate values during calculations. Finally, **Tensor Core Acceleration** will be explored on compatible NVIDIA GPUs to utilize specialized hardware for faster matrix computations.

The effectiveness of these optimization techniques will be evaluated using different matrix sizes. Performance will be measured using execution time, computational throughput (GFLOP/s), speedup, and numerical accuracy. The results will be compared against the basic CPU implementation, the basic GPU implementation, and NVIDIA's optimized cuBLAS library to understand the improvements achieved through the proposed techniques.

The expected outcome is a systematically optimized GPU matrix multiplication implementation that demonstrates how parallel processing, efficient memory management, and hardware-specific optimization techniques can improve computational performance. This project will also provide practical understanding of GPU architecture and the engineering principles involved in developing high-performance computing applications.